

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE FÍSICA

**Desenvolvimento de um Robô Móvel Autônomo e Controlado
por Bluetooth Baseado no ATmega328P**

**LUCAS FÜHR
GUILHER MENDES DARSKI
GONÇALO GOMES RUTKAY PEREIRA**

Porto Alegre - RS
2026

RESUMO

O trabalho a seguir apresenta o desenvolvimento de um robô com modos de operação autônomos e controlado remotamente em instâncias limite, inspirado em robôs aspiradores de pó, baseado no microcontrolador ATmega328P. O robô foi projetado para navegação em ambientes com desvio de obstáculos, a partir de leituras do sensor ultrassônico HC-SR04, oferecendo também a opção de ser controlado remotamente por meio do módulo Bluetooth HC-06. O chassi 2WD é montado com dois motores DC controlados por uma ponte H L298N. O firmware foi desenvolvido em linguagem Assembly para o Arduino Uno, implementando lógica de desvio de obstáculos com rotação progressiva e controle de velocidade via PWM. O projeto aborda conceitos fundamentais de sistemas embarcados, como temporização, interrupções, comunicação serial e controle de atuadores, estendendo-se também à montagem de um hardware conciso e autossuficiente. Apesar de possuir limitações em sua resiliência mecânica, a estrutura é capaz de sustentar o uso ativo sem perder suas funcionalidades em condições normais de operação.

1 INTRODUÇÃO

A robótica móvel autônoma e controlada está presente e é de extrema utilidade em diversas áreas, desde aplicações industriais, áreas militares e sistemas de transporte, até mesmo em ambientes domésticos, onde o uso de robôs aspiradores de pó se tornou um item comum em muitas casas. Sabendo dessas aplicações e na tentativa de assimilar os fundamentos de sistemas embarcados, nosso grupo propôs-se a desenvolver para este projeto uma versão simplificada e essencial de um robô autônomo. Tal robô móvel seria baseado no microcontrolador ATmega328P e capaz de explorar conceitos fundamentais como controle de motores DC, sensorização ultrassônica, modulação por largura de pulso (PWM) para controle de velocidade e comunicação sem fio via Bluetooth. O robô viria a possuir todas essas funcionalidades, aliadas a um baixo custo, programação de baixo nível e autonomia energética com baterias internas.

A motivação para este projeto reside na aplicação prática dos conceitos estudados na disciplina de Sistemas Embarcados. Percebemos que, mais do que a aplicação de conceitos isolados, nossas maiores dificuldades surgiram na integração das múltiplas partes funcionais dos componentes de hardware e, principalmente, no uso da linguagem de programação Assembly, que nos demandou atenção especial às suas limitações. Apesar disso, tais limitações nos possibilitaram um entendimento mais aprofundado do funcionamento do robô e seus componentes. Assim, o robô desenvolvido é capaz de operar em dois modos: autônomo, utilizando o sensor ultrassônico para detectar obstáculos e tomar decisões de navegação, e controlado remotamente via smartphone, utilizando um módulo Bluetooth para receber comandos de movimento. Este relatório documenta todas as etapas do projeto, desde a especificação dos componentes até a implementação do firmware e testes de validação, contribuindo para o aprendizado prático em

sistemas embarcados e robótica.

2 METODOLOGIA

A metodologia adotada para o desenvolvimento baseia-se em cinco etapas: prova de conceito, onde buscou-se componentes capazes de realizar a tarefa proposta e softwares que se adequassem aos casos que nosso problema inicial apresentava; teste dos componentes, onde não apenas testamos sua funcionalidade, mas também desenvolvemos um software próprio que permitiu sua utilização; montagem do projeto, na qual as partes elétricas, eletrônicas e peças sólidas (como as rodas e o chassi impresso em 3D) foram montadas de forma estável, constituindo um único carrinho robô; desenvolvimento de firmware, etapa em que integramos os códigos das etapas anteriores em uma única lógica coesa; e, por fim, testes do protótipo, onde ajustamos os últimos detalhes para o funcionamento pleno do projeto.

Para o desenvolvimento do projeto nos subdividimos dentro do próprio grupo entre três frentes de implementação distintas considerando as três vertentes principais do projeto: movimentação por meio da operação dos motores DC, detecção de obstáculos a curto alcance pelo sensor ultrassônico e recebimento e processamento de sinais através do módulo bluetooth. Uma vez que cada módulo tivesse sua lógica própria funcionando foram então despendidos esforços pelo grupo para integrá-los de maneira coesa e anexá-los ao carrinho.

2.1 Prova de Conceito

Para a prova de conceito, foi realizada uma ampla seleção de componentes que atendessem aos requisitos do projeto. Foram considerados fatores como disponibilidade, custo, facilidade de integração e adequação ao microcontrolador ATmega328P. A tabela 1 apresenta a lista completa de materiais utilizados, com suas respectivas funções e fontes de aquisição.

Tabela 1: Lista de componentes e materiais utilizados no projeto

Componente	Quantidade	Função	Fornecedor
Arduino Uno	1	Placa de desenvolvimento com microcontrolador ATmega328P	Laboratório
Chassi Arduino 2WD	1	Corpo do robô, contém dois motores DC acoplados, suporte para 4 pilhas AA e switch ON/OFF	Eletrogate
Ponte H Dupla L298N	1	Controle independente dos dois motores DC	Eletrogate
Pilhas AA 1,5V	4	Alimentação dos motores (circuito de potência)	Mercado local
Bateria 9V	1	Alimentação do Arduino Uno	Mercado local
Conector P4 com borne	1	Conexão entre bateria 9V e plug P4 do Arduino	Eletrogate
Módulo Ultrassônico HC-SR04	1	Sensor de distância para detecção de obstáculos	Eletrogate
Suporte Ultrassônico	1	Fixação do sensor ultrassônico ao chassi	Impressão 3D
Módulo Bluetooth HC-05	1	Comunicação sem fio para controle remoto	Eletrogate

Fonte: Elaborado pelos autores.

2.2 Teste dos Componentes

Para esta etapa, decidimos dividir o grupo para uma melhor eficiência. Aqui não estávamos apenas testando a funcionalidade dos componentes, verificando se não estavam quebrados de fábrica, mas também buscando desenvolver códigos que funcionassem e servissem de base para o firmware final. As três partes principais do protótipo, sensor ultrassônico, módulo Bluetooth e os dois motores, foram separadas, e para cada uma foi desenvolvido um pequeno circuito de testes. Pode-se ressaltar que, nessa etapa, os membros do grupo se dividiram para que, individualmente, cada um trabalhasse no teste e na montagem dos códigos e conteúdos.

2.2.1 Módulo Ultrassônico

O módulo ultrassônico HC-SR04 é um dos mais populares no mercado de prototipagem, por sua disponibilidade, mas principalmente por sua relativa simplicidade de implementação, onde o chip interno e o pequeno circuito integrado realizam a maioria das funções, restando externamente apenas a leitura do sinal de saída. Tal sensor possui um emissor de ondas ultrassônicas e um receptor, cujas funções são simplificadas para entradas e saídas binárias. Para sua operação, é necessário enviar um sinal para a porta TRIG, correspondente à ativação do emissor ultrassônico, e ler o sinal ECHO, correspondente ao receptor ultrassônico. O sinal de ECHO permanece ativo por um período proporcional ao tempo que a onda ultrassônica emitida leva para refletir em uma superfície e retornar ao sensor. Dessa forma, o tempo em que o sinal

ECHO permanece em nível alto corresponde diretamente à distância percorrida pela onda.

O código completo para a leitura do sensor ultrassônico, desenvolvido em Assembly para o ATmega328P, está disponível no repositório do projeto no GitHub. A implementação utiliza o Timer2 com prescaler de 256, resultando em uma resolução de aproximadamente 16 μ s por tick. A conversão para centímetros é realizada pela fórmula $cm = (TCNT2 \times 10) / 36$, considerando a velocidade do som no ar (343 m/s) e o fato de que a onda percorre o dobro da distância até o obstáculo (ida e volta).

2.2.2 Motores DC e Ponte H Dupla L298N

A tração do robô é feita por dois motores de corrente contínua dispostos de forma espelhada, um em cada lado do chassi, complementados por uma roda boba livre que fornece o terceiro ponto de apoio. Os motores DC são acionados de forma diferencial por meio da combinação dos sentidos de rotação das duas rodas motrizes através da alteração do sentido da corrente elétrica na eletrônica. O acionamento de cada motor é feito por uma ponte H, circuito formado por quatro chaves eletrônicas dispostas em "H" com o motor no centro. Ao fechar a chave superior de um lado e inferior do outro, a corrente atravessa o motor em um sentido padrão, e ao inverter o par de chaves acionado a corrente flui no sentido contrário, girando o motor para o lado oposto, sem inverter fisicamente os fios da bateria.

Além dessa reversão de polaridade sob demanda, o módulo L298N separa o circuito de potência dos motores, de corrente elevada, da lógica de baixa tensão do microcontrolador. Cada ponte recebe dois sinais digitais de entrada (os pinos IN), e é a combinação de níveis nesses dois pinos que determina o estado do motor: para frente, para trás, um sentido de giro, o sentido oposto, ou desligado em caso do PWM ser desconectado. No código, o controle de sentido foi isolado em quatro sub-rotinas curtas, cada uma responsável apenas por escrever a combinação correta de níveis no quatro pinos IN, usando `sbi` para configurar nível alto e `cbi` para configurar nível baixo. Os pinos foram mapeados de forma que PD2/PD3 comandem o motor esquerdo e PD4/PD5 o motor direito, com convenções próprias para determinar a direção de giro. A partir disso, `RUN_MOTORS` coloca as duas rodas para frente; `STOP_MOTORS` zera os quatro pinos; `TURN_RIGHT` e `TURN_LEFT` correspondentemente acionam as rodas alternadamente em rotações opostas, ocasionando um giro em torno do próprio eixo sem sair do lugar, o que só é possível pela roda boba.

A velocidade dos motores é controlada por PWM e os duty cycles utilizados no trabalho correspondem aos modos de velocidade. Esse sinal é gerado por hardware pelo Timer0 no caso do carrinho e é configurado em modo Fast PWM com saída no pino PD6. O prescaler de 64 cria uma frequência de aproximadamente 976 Hz, adequada para a operação dos motores. O ciclo de trabalho é definido pelo valor escrito em OCR0A, e um mesmo sinal de PD6 alimenta as duas entradas de habilitação da ponte (ENA e ENB), garantindo que as duas rodas girem

sempre na mesma velocidade.¹ As três marchas foram implementadas com um registrador de estado interno (SPEED_REG) que guarda a marcha atual (0, 1 ou 2); as sub-rotinas SPEED_UP e SPEED_DOWN incrementam ou decrementam esse índice, com saturação nos extremos e carregam o OCR0A com o valor correspondente a 35%, 63% e 100% de potência.

2.2.3 Comunicação com Módulo Bluetooth

O terceiro componente testado foi o envio de comandos via Bluetooth para o ATmega328P, com o intuito de instruir manualmente o robô a realizar ações específicas, sendo elas: aumentar e diminuir a velocidade (em três níveis), parar, iniciar movimento de ré e retomar movimento para frente com funcionamento do sensor ultrassônico. Optou-se por enviar os comandos através de um aplicativo móvel chamado *Serial Bluetooth Terminal*, no qual é possível digitar caracteres em uma interface de terminal, similar ao monitor serial da Arduino IDE. Para enviar os comandos, basta parear o módulo receptor Bluetooth HC-05 com o dispositivo (enquanto o LED estiver piscando) e, em seguida, conectá-lo no menu do aplicativo.

O módulo Bluetooth se comunica com o microcontrolador através do protocolo de comunicação USART, um tipo de comunicação serial, fazendo uso de dois pinos: RX (Receiver) e TX (Transmitter). Para o propósito do projeto, como precisávamos apenas enviar sinais do módulo para o ATmega328P, e nunca o contrário, foi necessário conectar apenas o pino TX do módulo ao pino RX da placa de desenvolvimento (PD0), considerando que nesse protocolo faz-se sempre a conexão cruzada. Evidentemente, também foram conectados os pinos GND e VCC aos pinos GND e 5V do Arduino Uno, respectivamente.

Inicialmente, foi desenvolvido um código preliminar para testar o recebimento dos comandos enviados pelo terminal no ATmega328P. Para a leitura do comando recebido, foram utilizados os pinos 0 a 5 da porta B, cada um deles conectado a um LED, representando os bits 0 a 5 do código em hexadecimal do caractere em questão.²

O código inicia com a configuração da comunicação USART entre o módulo e o ATmega328P, com a definição da taxa de transmissão de dados em 9600 bps (baud rate padrão de comunicação do HC-05) através do valor 103 carregado nos registradores UBRRL e UBR0H, o qual se relaciona com o baud rate através de uma fórmula descrita no código. Em seguida, é necessário setar os bits RXEN0 e RXCIE0 do registrador UCSR0B, para habilitar a recepção de dados pelo canal USART e a respectiva interrupção associada (ativada quando há dados a serem lidos no buffer de RX). Finalmente, são configurados no registrador UCSR0C o tamanho de caractere para 8 bits, o modo assíncrono e a desabilitação do modo de paridade.

Assim que a interrupção é acionada, o programa direciona-se para a sub-rotina de leitura

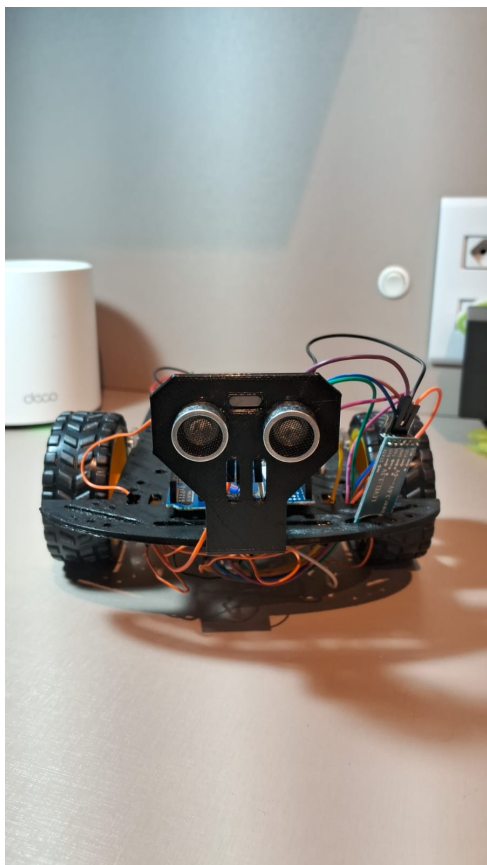
¹Em teoria, no nosso caso não foi o comportamento observado devido ao que podemos imaginar que sejam diferentes arquiteturas internas do motor ou desalinhamento da montagem, causando uma leve tendência ao carrinho para andar a esquerda devido a uma rotação maior da roda direita.

²Foram utilizados apenas 6 bits, ao invés de 8, devido à limitação de pinos disponíveis na porta B, o que não se mostrou um problema, dado que o objetivo era apenas detectar se esses bits correspondiam aos esperados para cada caractere.

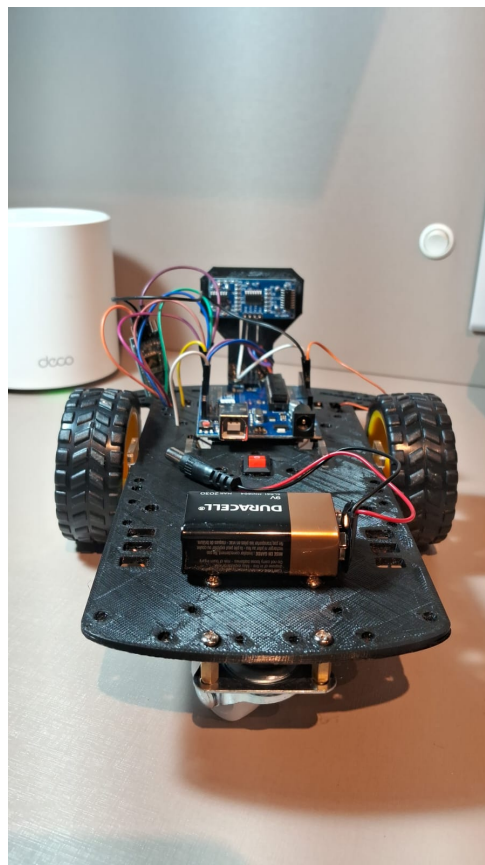
de dados do registrador UDR0. Nos primeiros testes, não foi possível exibir corretamente os valores nos LEDs, pois não se sabia que o comando do aplicativo não consistia apenas no byte do comando, mas também em bytes demarcadores de início e fim de mensagem. Por isso, foi implementado um filtro para desconsiderá-los. Após essa alteração, os LEDs exibiram os valores corretamente, o que foi confirmado com o auxílio de uma tabela de conversão de caracteres ASCII para hexadecimal.

2.3 Montagem do Projeto

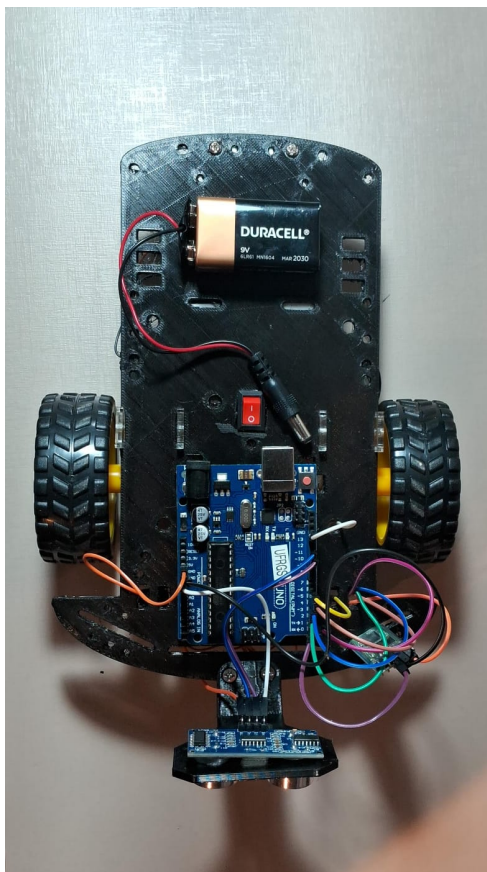
A estrutura do robô foi construída sobre um chassi principal impresso em 3D, ao qual os dois motores de corrente contínua foram fixados por meio de suportes próprios, parafusados à base. Cada motor possui dois terminais, aos quais foram soldados dois cabos que o conectam aos pinos de saída da respectiva ponte H. Convém observar que esses dois terminais não correspondem a um polo positivo e um negativo fixos: é a ponte H que inverte, sob demanda, a polaridade aplicada a eles, e é justamente essa inversão que faz o motor girar em um sentido ou no outro. Assim, qual dos dois cabos recebe a tensão positiva em um dado instante depende do estado momentâneo da ponte, e não de uma ligação permanente. Os demais componentes foram distribuídos entre as duas faces do chassi. Na face inferior, próximo aos motores, foram fixados o módulo da ponte H (L298N) e o suporte de 4 pilhas AA que alimenta os motores, ambos presos com fita. A alimentação dos motores pode ser ativada ou desativada manualmente através de um botão liga/desliga, o qual foi fixado em uma fenda no chassi e a cujos terminais foram soldados fios que se conectam ao positivo das pilhas (+6V) e à entrada de tensão dos motores da ponte H. Na face superior ficaram o sensor ultrassônico — montado em um suporte também impresso em 3D, parafusado ao chassi — além do Arduino UNO e da bateria de 9 V que o alimenta através da conexão P4, estes fixados com fita. O uso de fita na maior parte dos módulos eletrônicos reflete o caráter de protótipo da montagem, feita de forma incremental no laboratório, e teve a vantagem de permitir reposicionar e substituir componentes com facilidade ao longo dos testes. Vale registrar ainda que a alimentação foi mantida em duas fontes independentes: as pilhas, que suprem os motores através da ponte H, e a bateria de 9 V, dedicada ao Arduino.



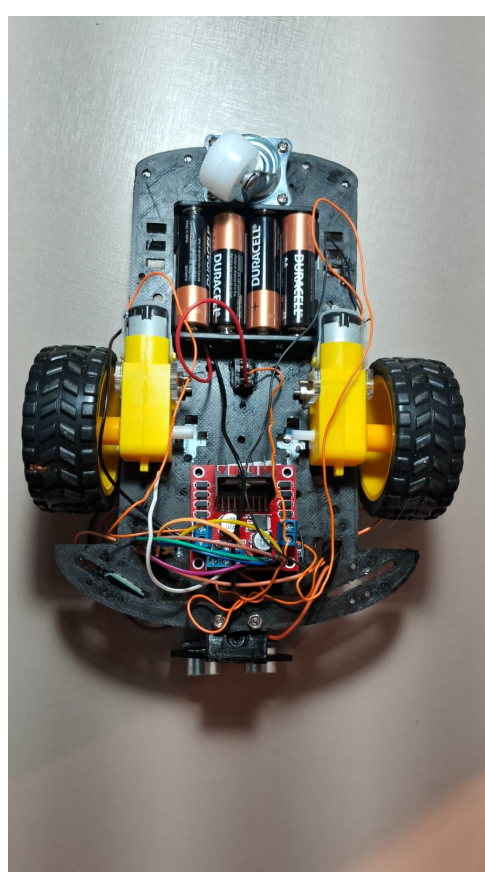
(a) Vista frontal



(b) Vista posterior



(c) Vista superior



(d) Vista inferior

Figura 1: Vistas da montagem do carrinho

2.4 Desenvolvimento de Firmware

Após serem desenvolvidos códigos para cada uma das três partes principais do robô e juntá-los em um único modelo funcional de carrinho, fazia-se necessário um código principal que integrasse as funcionalidades de cada módulo. Esse processo foi facilitado devido à nossa antecipação de possíveis problemas, pois mesmo em etapas anteriores já havia sido definido quais timers e portas seriam alocados para cada periférico, tornando a integração muito menos complicada.

O sistema pode ser dividido em dois blocos funcionais: os sensores e módulos de captação de informações (Bluetooth e sensor ultrassônico) e os atuadores, representados pelos motores esquerdo e direito. Essas mesmas sub-rotinas de motor são reaproveitadas pelos dois modos de operação. No modo autônomo, o laço principal mede a distância até um obstáculo com o sensor ultrassônico e, enquanto o caminho está livre, mantém os motores acionados no modo `RUNNING_FORWARD`; ao detectar algo mais próximo do que 10 cm, chama `STOP_MOTORS` e inicia uma sequência de desvio. A duração de cada giro é definida pelo `Timer1` em modo CTC: ele é zerado, ligado com prescaler de 1024 e gera uma interrupção de comparação (`INT_TIMER1_COMPA`) ao atingir a contagem correspondente a cerca de 400 ms, sinalizando o fim da rotação por meio de uma flag que a sub-rotina `WAIT_ROTATION` monitora. Um contador (`ROT_COUNT`) registra quantos desvios já foram feitos em sequência, encadeando os giros em uma ordem predeterminada: na primeira detecção, o robô gira 90 graus à esquerda; na segunda, realiza duas rotações de 90 graus à direita; na terceira, mais uma rotação à direita, retornando aproximadamente no sentido oposto da direção original. Caso encontre obstáculos em todas as direções, o robô permanece parado até que o caminho seja liberado.

A qualquer momento do código, caso haja caracteres recebidos via Bluetooth pela USART, é disparada a interrupção de recepção (`INT_RX_USART`), que consulta uma tabela de comandos armazenada na própria memória de programa (*lookup table*), por meio do registrador apontador Z, comparando um a um com o comando significativo recebido (após a filtragem). Assim que há uma equivalência, executa-se um salto indireto (`i jmp`), direcionando a execução para a sub-rotina de motor correspondente ao comando recebido, sendo cada um deles associado ao endereço da SR correspondente. Os comandos disponíveis são: **G** (GO) – inicia o movimento do robô para frente; **S** (STOP) – interrompe imediatamente o movimento, colocando o robô em estado de espera (`IDLE`); **B** (BACK) – inicia o movimento do robô para trás, no estado `RUNNING_BACKWARDS`; **+** (`SPEED_UP`) – aumenta a velocidade dos motores em um nível; **-** (`SPEED_DOWN`) – diminui a velocidade dos motores em um nível. A variável `CAR_STATE` gerencia a interação entre os modos autônomo e controlado, impedindo que o sensor atue e reative os motores no sentido para frente quando o robô está no estado `IDLE` ou `RUNNING_BACKWARDS`. Assim, tanto a navegação automática quanto o comando manual convergem para a mesma camada de acionamento, escrita uma única vez. De forma similar, a variável `SPEED_REG` armazena o modo de velocidade atual dos motores, permitindo incre-

mentar ou decrementá-lo (0 - 2).

A verificação do funcionamento foi realizada em etapas, do ambiente mais controlado ao mais realista. Primeiramente, em simulação com o `avr_sim`, cada sub-rotina foi executada isoladamente, conferindo no visualizador de registradores se os bits de PD2 a PD5 assumiam exatamente a combinação esperada em cada estado (avanço, parada e giros), e se OCR0A recebia o valor correto a cada troca de marcha via `SPEED_UP` e `SPEED_DOWN`. Em seguida, na protoboard, os motores foram acionados com a ponte H já alimentada pela bateria, verificando o sentido físico de rotação de cada roda. Quando uma roda girava no sentido oposto ao esperado, a correção foi feita invertendo seus dois fios no borne de saída da ponte, sem necessidade de recompilar o código. Por fim, com o módulo Bluetooth conectado, cada comando foi enviado individualmente para confirmar que acionava a função correta e que os dois modos operavam de forma independente. A duração do giro (o valor de `ROT_DELAY`) foi ajustada empiricamente nessa fase, observando o ângulo real percorrido pelo carrinho a cada rotação até chegar próximo dos 90 graus desejados.

O código pode ser acessado integralmente no seguinte endereço: https://github.com/Gongrp/Projeto_Embarcados/blob/main/FINAL/Final_bluetooth.asm

2.5 Testes do Protótipo

Por fim, como etapa final do projeto, o firmware foi carregado no Arduino e o protótipo foi testado. Ao conectar o cabo da bateria no Arduino, ligá-lo e colocar o modelo no chão, alguns problemas puderam ser notados imediatamente. Primeiro, um dos motores apresentava menos potência que o outro, fazendo com que o protótipo andasse em círculos. Em segundo lugar, a ponte H apresentava uma queda de tensão de 2V, fazendo com que os motores operassem ambos em uma potência mais baixa. Para resolver o primeiro problema, foi adquirido um motor novo, já que a falha decorria de uma parte defeituosa na parte traseira do motor, devido ao aquecimento com o ferro de solda. O segundo problema foi resolvido ao unificarmos a alimentação: a ponte H, ambos os motores e o Arduino Uno passaram a ser alimentados pela bateria de 9V.

3 RESULTADOS

Os resultados obtidos demonstram o funcionamento satisfatório do robô em ambos os modos de operação, com as devidas ressalvas quanto as limitações do carrinho. No modo autônomo, o robô foi capaz de detectar obstáculos a uma distância de até 10cm, executando a sequência de rotações programada: primeira rotação à esquerda, segunda e terceira rotações à direita, conforme a lógica implementada para explorar o ambiente. A medição ultrassônica apresentou boa precisão em distâncias na ordem de grandeza de dez centímetros, com leituras estáveis realizadas a cada intervalo, conforme recomendado no datasheet do sensor. A imple-

mentação do Timer1 com prescaler de 1024 e interrupção por comparador proporcionou um controle razoável dos delays de rotação, ajustados para aproximadamente 400ms³.

No modo de controle remoto via Bluetooth, os comandos de movimento foram recebidos corretamente pela USART do microcontrolador, com a tabela de lookup implementada em Assembly direcionando adequadamente para as sub-rotinas de ação correspondentes. O controle de velocidade, implementado através do PWM do Timer0, permitiu a regulação em três níveis (lento, médio e rápido), com os valores de duty cycle configurados para 205, 230 e 255 respectivamente. O sistema operou com relativa eficácia com alimentação separada (4 pilhas AA para os motores e 1 bateria de 9V para o Arduino), evitando interferências elétricas entre os circuitos de potência e controle, embora mais tarde foi constatado um problema relativo a queda de tensão significativa ocasionada pela ponte H, de 2V, o que prejudicou a implementação da alimentação com pilhas em conjunto com a manutenção de uma corrente mínima de operação para os motores.⁴

Quanto ao consumo de recursos, o firmware em Assembly mostrou-se eficiente, ocupando apenas uma pequena fração da memória flash disponível e executando as rotinas com latência mínima, atendendo aos requisitos de operação remota. Os testes de desvio de obstáculos realizados em diferentes superfícies mostraram que a aderência das rodas e o peso do chassi influenciaram na execução precisa das rotações, sugerindo a necessidade de ajustes futuros nos parâmetros de temporização. Além disso, possíveis melhorias quanto ao alinhamento dos motores e rodas e disposição dos componentes no carrinho certamente contribuiriam positivamente para correção das dificuldades de andar em linha reta. A inclusão de um suporte para pilhas maior com capacidade superior também serviria como resolução satisfatória para as necessidades dos motores e a impressão da eletrônica utilizada em uma PCB beneficiaria enormemente o projeto em redução de fios e constantes problemas com desconexão.

4 CONCLUSÃO

Apesar de uma série de problemas nas últimas semanas de montagem, como erros ao utilizarmos a bateria e falhas menores como o comando Bluetooth de parada, o projeto do carrinho aspirador de pó apresentou resultados satisfatórios, consistindo em um protótipo funcional com funções autônomas e controladas. A integração entre sensor, módulo Bluetooth e motores mostrou-se eficiente no ATmega328P, de modo que o protótipo final demonstrou ser capaz de realizar as metas propostas no início do trabalho. Pode-se acrescentar que, apesar de o projeto final apresentar problemas e limitações, foi possível perceber que tais dificuldades decorriam de falhas pontuais de hardware ou de funcionalidades que estavam fora do escopo inicial do

³A determinação do tempo de rotação foi obtida através de tentativa e erro e por vezes se mostrou o suficiente para implementar rotações em ângulos retos. Fatores como o atrito da roda boba e outras imperfeições não permitem assegurar sempre uma rotação perfeita em 90°

⁴Para contornar esse predicamento, os valores de duty cycle correspondentes às marchas do carrinho foram aumentados para garantir uma alimentação média constante de ao menos 3V.

trabalho.

Dentre as principais dificuldades enfrentadas, destacam-se o gerenciamento dos timers para múltiplas funcionalidades simultâneas e a divisão de tensão e corrente entre os motores, a ponte H e o Arduino, o que, nos estágios finais, chegou a comprometer o desempenho das pilhas e baterias. Nos últimos instantes em que estávamos realizando o trabalho, diversas ideias de melhorias puderam ser pensadas, dentre elas a confecção de uma PCB, melhorias no chassi, a integração do sensor ultrassônico com a quantidade de rotação e, por fim, o ajuste detalhado dos PWMs para cada motor, de modo a compensar suas diferenças de fábrica e garantir velocidades equivalentes. Tais ideias ficam como sugestões de melhoria para projetos futuros, sejam eles robôs mais autônomos ou robôs capazes de resolver labirintos. O nosso projeto se mostrou uma excelente base para futuras melhorias, demonstrando a necessidade do conhecimento aprofundado e fundamental de computação e de integração hardware-software.

Referências

- [1] LAST MINUTE ENGINEERS. **In-Depth: Interface L298N DC Motor Driver Module with Arduino**. Disponível em: <https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/>. Acesso em: 3 jul. 2026.
- [2] BLOG DA ROBÓTICA. **Como utilizar o Módulo Bluetooth HC-05 com Arduino**. 2023. Disponível em: <https://www.blogdarobotica.com/2023/02/13/como-utilizar-o-modulo-bluetooth-hc-05-com-arduino/>. Acesso em: 3 jul. 2026.
- [3] MAKERHERO. **Guia Completo do Sensor Ultrassônico HC-SR04 com Arduino**. Disponível em: <https://www.makerhero.com/blog/sensor-ultrassonico-hc-sr04-ao-arduino/>. Acesso em: 3 jul. 2026.
- [4] ELETROGATE. **Robô que evita obstáculos com Arduino**. Disponível em: <https://blog.eletrogate.com/robo-que-evita-obstaculos-com-arduino/>. Acesso em: 3 jul. 2026.
- [5] ATMEL CORPORATION. **ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet**. Atmel, 2015. Disponível em arquivo PDF. Acesso em: 3 jul. 2026.
- [6] ROBÓTICA PARANÁ/SKETCHFAB. **Modelo 3D do chassi do carrinho**. 2024. Disponível em: <https://sketchfab.com/3d-models/kite-chassi-2wd-13f86020ae754a48ab2567138b8d31cb>. Acesso em: 3 jul. 2026.

- [7] AAKAR LAB/PRINTABLES. **Modelo 3D do suporte do sensor ultrassônico.** 2022. Disponível em: <https://www.printables.com/model/308785-ultrasonic-holder-new-design>. Acesso em: 3 jul. 2026.